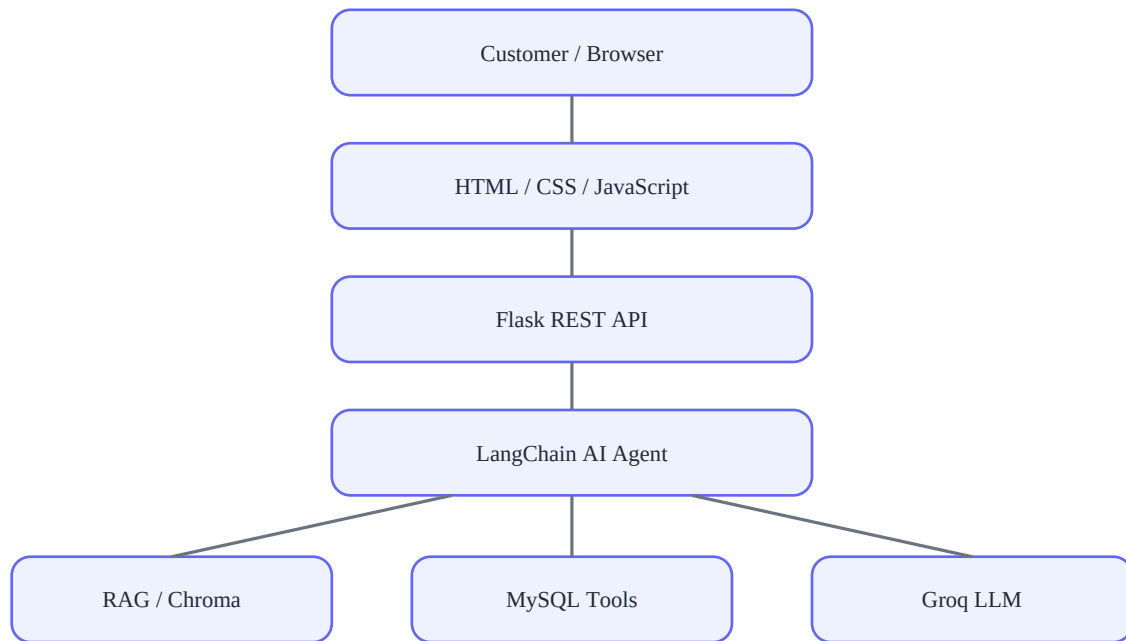


AI Customer Support Agent

Technical Project Showcase

LLM Agent • RAG • Chroma • MySQL • LangChain • Flask • HTML/CSS/JavaScript



Author: Mohammed Moshin

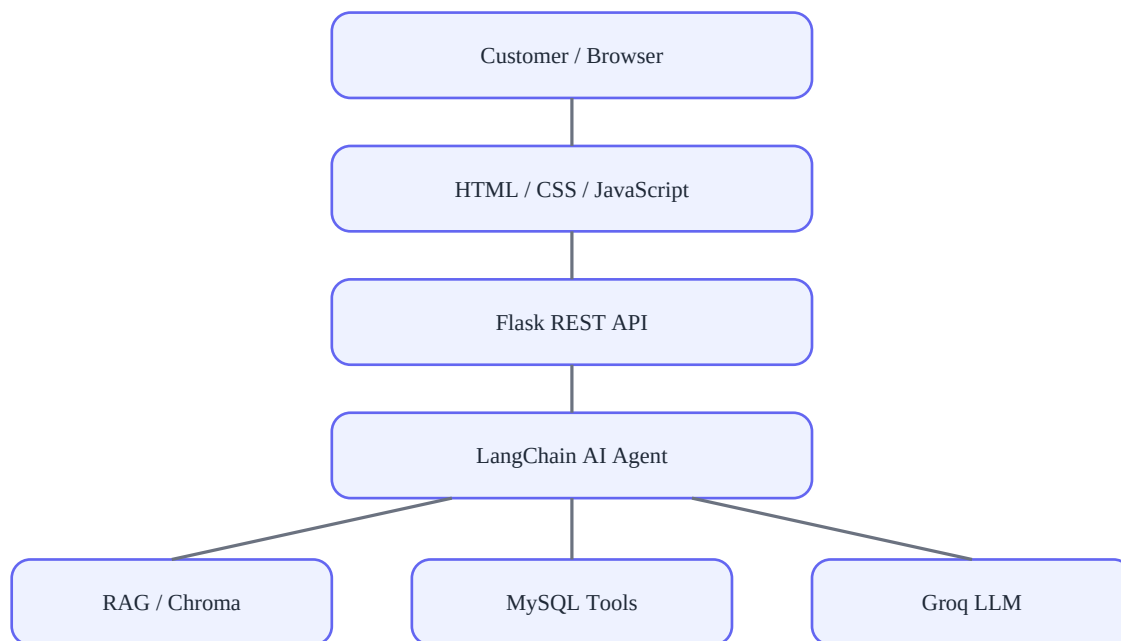
Project goal: Build a focused e-commerce support agent that combines unstructured policy retrieval with structured database tools.

1. Project Overview

This application is a practical AI customer-support system. Instead of acting as a general chatbot, the LLM is used as an orchestration layer that selects specialized tools and retrieves the appropriate information.

The application supports policy questions through RAG and customer/order questions through MySQL tools.

2. System Architecture



Frontend JavaScript sends a JSON request to Flask at **POST /ask**. Flask invokes the LangChain agent, which can call RAG or MySQL tools before producing the final answer.

3. RAG Pipeline

Policy documents are loaded, split into chunks, converted into vector representations with **sentence-transformers/all-MiniLM-L6-v2**, and stored in Chroma. A customer question is embedded with the same model and compared semantically against the stored vectors to retrieve relevant context.

RAG is for unstructured knowledge; MySQL is for structured transactional data.

4. Tools

Tool	Purpose
search_knowledge_base	Retrieve company policies/documentation from Chroma.
check_order_status	Retrieve order status and delivery information.
get_order_items	Retrieve products, quantities, and prices.
get_payment_details	Retrieve payment method, status, date, and amount.
get_support_tickets	Retrieve support ticket information and status.

5. Prompt Engineering

The system prompt was deliberately made strict. The assistant is restricted to e-commerce customer-support topics and is instructed not to answer unrelated general-knowledge questions.

The prompt also prevents the agent from guessing missing IDs or inventing customer-specific information. Tool results are treated as the source of truth.

6. Example Flows

Question	Expected behavior
How long do I have to get my money back?	Use RAG / knowledge base
What is the status of ORD1001?	Use order-status tool
What products are in ORD1001?	Use order-items tool
How did I pay for ORD1005?	Use payment tool
Status of ORD1001 and what products are in it?	Use multiple tools
What is the status of my order?	Ask for order ID
Who is the Prime Minister of Pakistan?	Decline as out of scope

7. Technology Stack

Layer	Technology
LLM	Groq-hosted model
Agent	LangChain
Embeddings	Hugging Face all-MiniLM-L6-v2
Vector DB	Chroma
Database	MySQL
Backend	Flask REST API
Frontend	HTML, CSS, JavaScript
API communication	Fetch API + Flask-CORS

8. Frontend / Backend Separation

The development setup separates the frontend from the backend. Live Server hosts the UI while Flask exposes the AI endpoint. This makes the browser-to-API boundary explicit and provides a clean foundation for later deployment.

```
Browser :5500 → fetch() → Flask :5000 → ask_agent() → RAG/MySQL → LLM → JSON → Browser
```

9. Security & GitHub

API keys and database passwords belong in a local .env file and must not be committed. The virtual environment, Python cache, and generated Chroma database should also be excluded from version control.

```
.env  
venv/  
__pycache__/  
*.pyc  
vector_db/
```

10. Learning Outcomes

The project demonstrates LLM applications, agents, tool calling, RAG, embeddings, vector databases, semantic search, SQL, prompt engineering, Flask APIs, frontend/backend integration, CORS, and basic hallucination/scope controls.

11. Future Improvements

Conversation memory, authentication, ticket creation/update tools, streaming responses, automated tests, retrieval evaluation, monitoring, Docker, and production deployment are natural next steps.

End of Report